Project 13: Steganography Tool for Image Data Hiding

Name: Mohammed Fadil PK

Domain: Cybersecurity / Digital Forensics

Date: February 26, 2026

1. Introduction

Steganography is the practice of concealing a file, message, image, or video within another file. In digital forensics and secure communications, it is often used to bypass traditional detection mechanisms. This project focuses on building a custom Python application that utilises the Least Significant Bit (LSB) method to embed secret text within image files without visibly altering the image.

2. Abstract

The "Steganography Tool" is a GUI-based desktop application developed in Python. It allows users to upload an image, embed a secret text message into the image's pixel data, and save the modified image. Furthermore, it includes an extraction feature that can scan an altered image and recover the hidden message.

3. Tools & Technologies Used

- **Operating System:** Kali Linux.
- **Language:** Python 3.
- **Libraries:** tkinter (for the GUI), Pillow (for image processing), stepic (for LSB steganography encoding/decoding).

4. Steps Involved in Building the Project

Step 1: Environment Setup & Library Installation

- Configured the Python environment and installed necessary dependencies (Pillow and stepic) to handle image manipulation at the byte level.

Step 2: Developing the Graphical User Interface (GUI)

- Utilised the tkinter library to build a user-friendly interface.
- Implemented interactive buttons and file dialogues to allow users to easily browse and select target images (.png format) from their local system.

Step 3: Implementing the Encoding (Hiding) Logic

- Wrote the hide_data() function, which prompts the user for a secret string.
- Used the stepic.encode() method to convert the string into bytes and embed it into the Least Significant Bits of the image pixels, ensuring the visual integrity of the image remains intact.

Step 4: Implementing the Decoding (Extraction) Logic

- Created the extract_data() function to reverse the process.
- Used stepic.decode() to scan the uploaded image, extract the hidden byte data, convert it back into a readable string, and display it via a pop-up alert box.

5. Results

Screenshot: Successful Data Extraction

Figure 1: *Execution of the Steganography Tool demonstrating the successful extraction of the hidden message ("Fadil was here") from the modified cover image.*

6. Conclusion

This project successfully demonstrates the principles of steganography and its application in digital security. The tool provides a practical, easy-to-use interface for both embedding and extracting hidden data. Understanding these techniques is crucial for digital forensics investigators who must detect and recover hidden payloads or covert communications during incident response.